TASK 25 - AWS ELASTIC BEANSTALK MIGRATION: APPLICATION PROVISIONING AND INFRASTRUCTURE CONFIGURATION

Objective

1. Study the architecture of AWS Elastic Beanstalk for automated application and environment orchestration.
2. Create an Elastic Beanstalk application container and provision a managed Python environment.
3. Access and verify the application deployment using the generated DNS URL.
4. Configure application load balancer settings for traffic distribution.
5. Verify the automatic provisioning and launching of underlying Amazon EC2 compute resources.

1. Elastic Beanstalk Basics

AWS Elastic Beanstalk is a managed service that simplifies the deployment and scaling of web applications and services developed in Python and other languages, handling provisioning, load balancing, auto-scaling, and health monitoring automatically.

Key concepts:

Term	Meaning
Application	A logical container for Elastic Beanstalk components, including environments, versions, and configurations.
Environment	A collection of AWS resources running a specific version of your application code.
Platform	The managed runtime stack (e.g., Python) provided to execute application code.
Load Balancer	A service that distributes incoming web traffic across multiple backend EC2 instances.

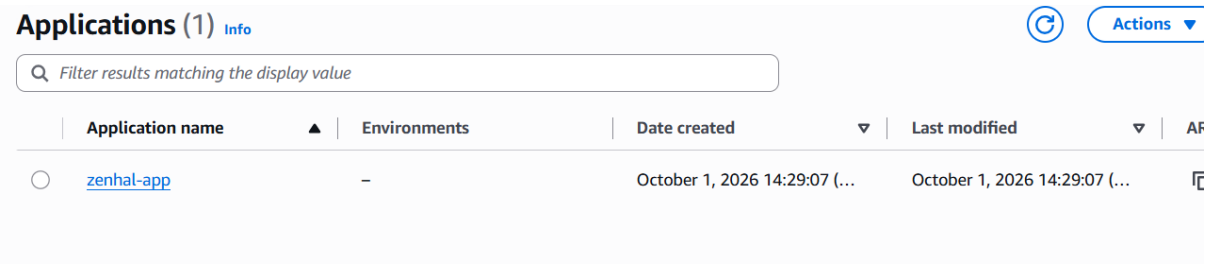
Term	Meaning
Environment Health	A live indicator (e.g., OK / Green) monitoring the operational state of the environment.

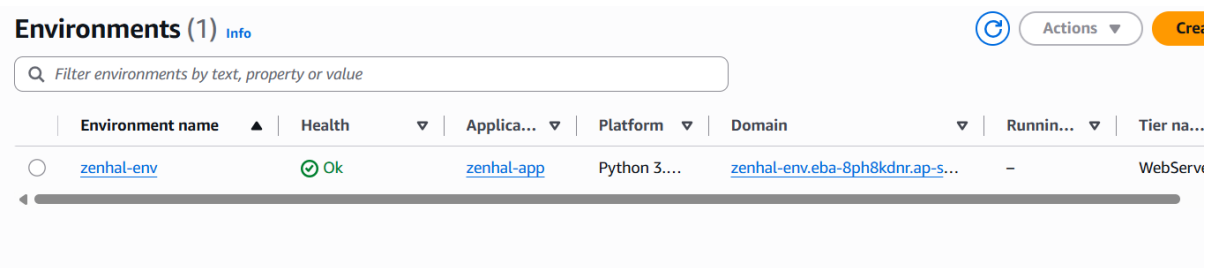
2. Application and Environment Creation

An Elastic Beanstalk application and environment were created through the AWS console to establish a managed hosting architecture.

Steps:

1. Navigate to the AWS Elastic Beanstalk console and create a new application named zenhal-app.
2. Create a new environment within the application using the following parameters:
    - **Environment Name**: zenhal-env
    - **Platform**: Python
    - **Deployment Mode**: Standard (EC2-backed web server environment)
3. Wait for the environment provisioning process to complete and confirm that the environment health status displays as **OK (Green)**.



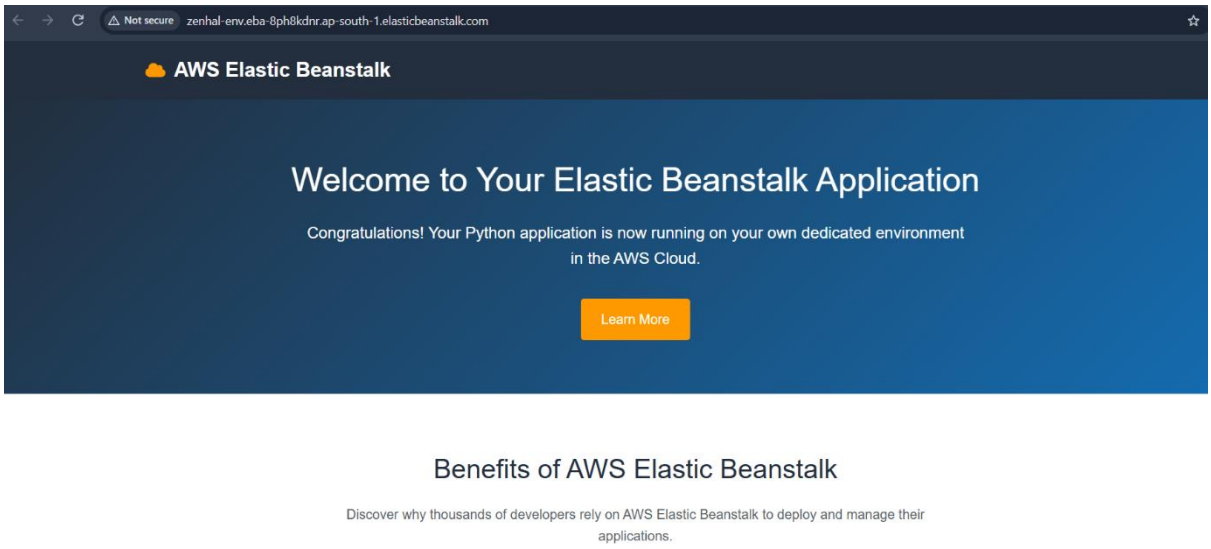


3. Application Access via DNS URL

Once the environment was successfully initialized, the deployed application was accessed to verify connectivity.

Steps:

1. Locate the automatically generated environment URL (DNS URL) in the Elastic Beanstalk console dashboard.
2. Open the DNS URL in a web browser to confirm that the default sample application page loads successfully.

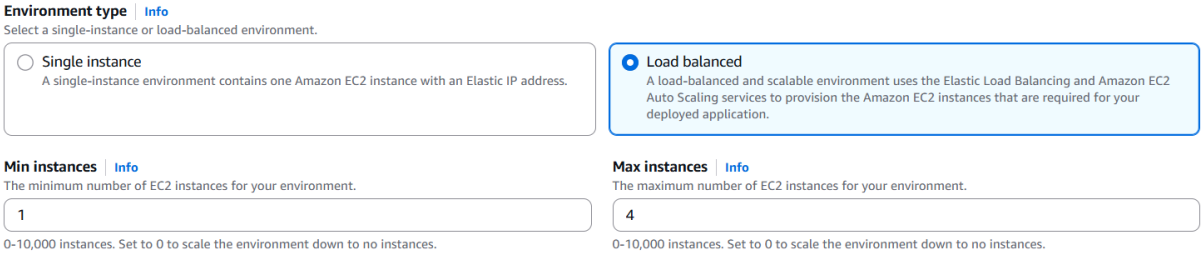


4. Load Balancer Configuration

To manage incoming traffic routing effectively, load balancer options were configured for the environment.

Steps:

1. Navigate to the **Configuration** section of your environment dashboard.
2. Select and edit the **Load balancer** settings card.
3. Ensure an Application Load Balancer is selected and listener rules/ports (such as HTTP port 80) are properly mapped.
4. Save the configuration changes to update the environment stack.

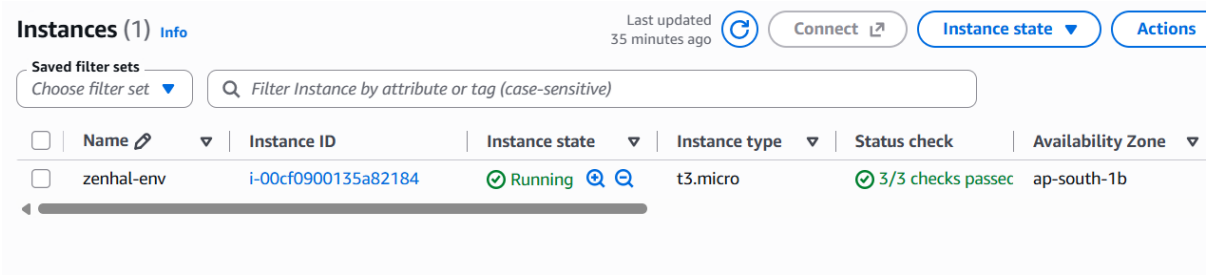


5. Automatic Instance Launch

Configuring the environment and load balancer automatically triggered the provisioning of underlying computing hardware.

Observation & Verification:

1. Access the **Amazon EC2 Console** and navigate to the **Instances** page.
2. Observe that a new EC2 instance has been automatically launched for the environment (e.g., instance i-00cf0900135a82184 running on type t3.micro).
3. Verify that the instance state is **Running** and all status checks have passed (3/3 checks passed), confirming full operational readiness.



Conclusion

This task successfully demonstrated the complete workflow of deploying an application on AWS Elastic Beanstalk: creating the application container, provisioning a Python environment, accessing the application via its DNS URL, configuring the load balancer, and verifying that the underlying EC2 compute instance was automatically launched and verified as healthy.